RELACIONAMENTOS ENTRE OBJETOS

Prof. André Backes

Relacionamentos entre objetos

- Um relacionamento ocorre quando um objeto conhece, depende, usa ou contém outro objeto em seu funcionamento.
- Os objetos não existem de forma totalmente isolada
  - Eles cooperam entre si para realizar tarefas mais complexas
  - Dividem responsabilidades
  - Formam uma rede de interações

Relacionamentos entre objetos

- Imagine um sistema simples de biblioteca:
  - A classe Livro representa os livros disponíveis.
  - A classe Pessoa representa os usuários.
  - A classe Empréstimo representa a relação entre uma pessoa e um livro emprestado.

```
class Livro {
    ...
};

class Pessoa {
    ...
};

class Emprestimo {
    ...
};
```

Relacionamentos entre objetos

- Os objetos dessas classes se relacionam naturalmente
  - Uma Pessoa pode tomar um Livro emprestado
  - Essa ação é registrada por um objeto da classe Empréstimo
  - Empréstimo referencia tanto a pessoa quanto o livro envolvidos.
- Esses vínculos são os relacionamentos entre os objetos.

```
class Livro {
    ...
};

class Pessoa {
    ...
};

class Emprestimo {
    ...
};
```

Relacionamentos entre objetos

- Também podem ser vistos como uma forma de agrupamento de dados
  - Semelhante ao que acontece com arrays e structs.
    - arrays: elementos homogêneos (do mesmo tipo)
    - structs: diferentes tipos de dados sob uma mesma estrutura

```
class Carro {
private:
    Motor motor;
    Roda rodas[4];
    PainelDeControle painel;
    bool motorLigado;
};
```

Relacionamentos entre objetos

- Relacionamentos são representados com
  - Variáveis membro
    - Objetos dentro de outros objetos
  - Ponteiros
  - Referências
- Eles conectam objetos em uma rede lógica
  - Dados e comportamentos são agrupados e também interagem entre si
  - Reflete melhor as interdependências do mundo real

```
class Carro {
private:
    Motor motor;
    Roda rodas[4];
    PainelDeControle painel;
    bool motorLigado;
};
```

Relacionamento x Herança

Herança

- Relações do tipo "é-um"
  - Um gato "é-um" animal.
  - Representação clara de hierarquias.
  - Facilita a implementação de polimorfismo.
  - Alterar a superclasse afeta as subclasses (forte acoplamento).
  - Subclasses podem ter que implementar métodos dos quais não precisam.

Relacionamento

- Relações do tipo "parte-de", "tem-um", "usa-um", "depende-de"
  - O motor é "parte-de" um carro; a biblioteca "tem-um" livro, etc
  - Maleável e fácil de modificar posteriormente
  - Promove maior reutilização de códigos.
  - Pode resultar em estruturas de objetos complexas.
  - Problemas com dados duplicados em objetos diferentes.

Tipos de relacionamentos

- Existem 4 formas em que os objetos podem se relacionar
  - Composição
  - Agregação
  - Associação
  - Dependência

COMPOSIÇÃO

Composição

- Uma classe inclui objetos de outras classes como parte de sua estrutura interna.
  - Relacionamento conhecido como “tem um” ou “é composto de”.
    - Herança define um relacionamento “é um” entre classes
  - Expressa que um objeto maior é formado por objetos menores, cada um com responsabilidades específicas.

Composição

- Declara objetos de uma classe dentro de outra classe
  - Promove um acoplamento mais fraco e maior flexibilidade na construção de sistemas complexos.
  - Ciclo de vida dependente: objetos membros são criados quando o objeto composto é instanciado e destruídos quando ele é destruído

Composição

- Relação unidirecional entre objetos
  - O objeto maior conhece e controla seus componentes
  - Os componentes geralmente não conhecem o objeto que os contém
    - Os componentes não mantêm referências ao objeto maior
    - Isso evita o surgimento de dependências circulares e facilita a manutenção.
  - Controle centralizado pelo objeto composto
  - Os componentes não precisam saber sobre o contexto onde são usados
    - Isso permite que eles sejam reutilizados em diferentes composições.

Composição | Exemplo

- Classes Motor e Roda
  - Classes que modelam partes específicas de um carro
  - Funcionam de forma independente uma da outra

```
// Componente Motor
class Motor {
public:
    void ligar() const { cout << "Motor: ligado\n"; }
    void desligar() const { cout << "Motor: desligado\n"; }
};

// Componente Roda
class Roda {
private:
    int posicao;
public:
    Roda(int idx) : posicao(idx) {}
    void rodar() const { cout << "Roda " << posicao << ": girando\n"; }
};
```

Composição | Exemplo

- Classe Carro
  - Contém atributos objetos da classe Motor e Roda
  - Suas funções membro são responsáveis por gerenciar o funcionamento e o tempo de vida das demais classes
  - Relação unidirecional

```
// Classe composta Carro
class Carro {
private:
    Motor motor;
    Roda rodas[4];
    bool motorLigado;

public:
    Carro() : rodas{Roda(1), Roda(2), Roda(3), Roda(4)}, motorLigado(false) {}

    void ligar() {
        motor.ligar();
        motorLigado = true;
    }

    void andar() {
        if (motorLigado) {
            for (const auto& roda : rodas) {
                roda.rodar();
            }
        } else {
            cout << "O carro está desligado. Ligue o motor primeiro.\n";
        }
    }

    void desligar() {
        motor.desligar();
        motorLigado = false;
    }
};
```

Composição

- Possibilita modelar estruturas complexas, compostas por partes que trabalham em conjunto
  - APIs Gráficas: Um Janela contém Botão, Menu ...
  - Jogos: Um Personagem é composto por Arma, Inventario, Atributos...
  - Sistemas Financeiros: Uma Carteira contém Ativo, Transação...
  - Controle de Hardware: Um Robô possui Sensor, Atuador, Controlador...

Composição

- Facilidade de expandir uma classe
  - Sem precisar alterar profundamente sua estrutura ou recorrer a herança
- Vamos adicionar um componente
  - Classe PainelDeControle
  - Será responsável por exibir informações sobre o estado do carro

```
// Componente PainelDeControle
class PainelDeControle {
public:
    void exibirStatus(bool motorLigado) const {
        if (motorLigado) {
            std::cout << "Painel: motor está LIGADO\n";
        } else {
            std::cout << "Painel: motor está DESLIGADO\n";
        }
    }
};
```

Composição

- Basta incluir como um objeto membro da classe Carro e chamar seus métodos nos lugares corretos
  - Sem necessidade de alterar as outras classes

```
// Classe composta Carro
class Carro {
private:
    Motor motor;
    Roda rodas[4];
    PainelDeControle painel; // novo componente
    bool motorLigado;

public:
    Carro() : rodas{Roda(1), Roda(2), Roda(3), Roda(4)}, motorLigado(false) {}

    void ligar() {
        motor.ligar();
        motorLigado = true;
        painel.exibirStatus(motorLigado);
    }

    void andar() { ... }

    void desligar() {
        motor.desligar();
        motorLigado = false;
        painel.exibirStatus(motorLigado);
    }
};
```

Composição

- A composição encapsula comportamentos e dados
  - Baixo acoplamento
  - Alta coesão
  - Promove maior reutilização de códigos
  - As classes mantêm suas implementações separadas e com responsabilidades bem definidas

```
// Classe composta Carro
class Carro {
private:
    Motor motor;
    Roda rodas[4];
    PainelDeControle painel; // novo componente
    bool motorLigado;

public:
    Carro() : rodas{Roda(1), Roda(2), Roda(3), Roda(4)}, motorLigado(false) {}

    void ligar() {
        motor.ligar();
        motorLigado = true;
        painel.exibirStatus(motorLigado);
    }

    void andar() { ... }

    void desligar() {
        motor.desligar();
        motorLigado = false;
        painel.exibirStatus(motorLigado);
    }
};
```

AGREGAÇÃO

Agregação

- Relacionamento entre objetos contendo conexão do tipo “tem um”
  - Semelhante à composição
  - Porém, o relacionamento é mais fraco e flexível
  - Permite que os objetos envolvidos tenham ciclos de vida independentes

Agregação

- Uma classe mantém referências ou ponteiros para objetos que ela utiliza
  - Essa classe não cria nem destrói automaticamente esses objetos
  - Os objetos agregados mantêm sua identidade e autonomia
- É usada quando queremos compartilhar um mesmo objeto entre várias classes sem duplicá-lo

Agregação

Composição

- O objeto componente faz parte do objeto que o contém
- Os objetos componentes são criados e destruídos junto com o objeto maior

Agregação

- O objeto componente pode existir de forma autônoma, fora da classe que o utiliza
- Os objetos agregados são fornecidos de fora e podem ser compartilhados por diferentes objetos

Agregação

- Imagine um sistema de universidade:
  - A classe Professor representa um professor.
  - A classe Departamento representa um departamento acadêmico.
  - Um professor pertence a um departamento, mas seu ciclo de vida não depende do departamento
  - O professor pode mudar de departamento ou existir mesmo que o departamento seja removido

Agregação | Exemplo

- Duas classes implementadas: Professor e Turma
- Turma possui
  - Uma string (disciplina ministrada)
  - Um ponteiro para Professor (quem ministra a disciplina)

```
// Classe Professor
class Professor {
private:
    string nome;
public:
    Professor(const string& n) : nome(n) {}

    void apresentar() const {
        cout << "Professor: " << nome << "\n";
    }
};

// Classe Turma que agrega um Professor
class Turma {
private:
    string disciplina;
    const Professor* professor;

public:
    Turma(const string& d, const Professor* p)
        : disciplina(d), professor(p) {}

    void apresentar() const {
        cout << "Turma de " << disciplina << "\n";
        professor->apresentar();
    }
};
```

Agregação | Exemplo

- Professor pode ser compartilhado por várias turmas diferentes
  - É criado fora da classe Turma e passado como ponteiro
  - Sua existência é independente da turma
  - Quando a Turma deixa de existir, o Professor permanece e pode ser usado novamente.

```
int main() {
    Professor prof("Dr. André");
    Turma turma1("Programação", &prof);
    Turma turma2("Estrutura de Dados", &prof);

    turma1.apresentar();
    turma2.apresentar();

    return 0;
}
```

Agregação

- A agregação define relacionamentos entre objetos que cooperam entre si, sem que um dependa da existência do outro
  - Facilita a reutilização e o compartilhamento de componentes/recursos
  - Baixo acoplamento
  - As classes mantêm suas implementações separadas e com responsabilidades bem definidas

ASSOCIAÇÃO

Associação

- Relacionamento entre objetos contendo conexão do tipo “usa um”
  - Representa a ideia de que um objeto conhece outro e pode interagir com ele, sem necessariamente conter ou controlar seu ciclo de vida
  - Uma classe armazena uma referência ou ponteiro para outro objeto, com o propósito de poder acessar seus métodos e dados

Associação

- Composição e agregação
  - Descrevem relações mais fortes
  - Um objeto possui outros como partes integrantes ou dependentes
- Associação
  - Indica apenas que há um vínculo de uso ou conhecimento entre objetos
  - Permite modelar interações e colaborações entre objetos que precisam trocar informações ou coordenar comportamentos
  - Não há um relacionamento implícito de todo/parte

Associação

- A associação expressa que uma classe usa serviços de outra
  - Os objetos existem independentemente uns dos outros
  - Sem controle sobre a sua existência ou propriedade
  - Baixo acoplamento entre partes do sistema

Associação

- A relação entre as classes pode ser
  - Unidirecional: apenas uma classe conhece a outra
  - Bidirecional: ambas as classes sabem que estão associadas

Associação

- Relação unidirecional
  - Um aluno sabe em que curso está matriculado, mas o curso pode não saber necessariamente todos os alunos.
- Relação bidirecional
  - O médico atende muitos pacientes e deles obtêm o seu salário
  - O paciente pode consultar vários médicos para curar a sua enfermidade e cuidar da sua saúde

Associação | Exemplo

- Duas classes implementadas: Curso e Aluno
  - O Aluno apenas aponta para o Curso
    - O ponteiro representa a associação entre as classes
    - Não propriedade
  - Quando Aluno chama curso->exibir(), ele utiliza serviços do Curso
    - Não controla seu ciclo de vida

```
// Classe Curso
class Curso {
private:
    string nome;
public:
    Curso(const string& n) : nome(n) {}

    void exibir() const {
        cout << "Curso: " << nome << "\n";
    }
};

// Classe Aluno associada ao Curso
class Aluno {
private:
    string nome;
    const Curso* curso;

public:
    Aluno(const string& n, const Curso* c) : nome(n), curso(c) {}

    void apresentar() const {
        cout << "Aluno: " << nome << "\n";
        curso->exibir();
    }
};
```

Associação | Exemplo

- Os objetos são criados de forma independente
  - Se o Curso for destruído, o aluno continua a existir e pode usar os serviços de outro curso
  - Apenas o ponteiro deve ser tratado com cuidado para evitar acessos inválidos

```
int main() {
    Curso curso1("Matemática");
    Aluno aluno1("Lucas", &curso1);

    aluno1.apresentar();
    return 0;
}
```

Associação

- A diferença entre agregação e associação está mais na semântica do que na sintaxe

Propriedade	Composição	Agregação	Associação
Tipo de relacionamento	Todo/parte	Todo/parte	Caso contrário, não relacionado
Membros podem pertencer a várias classes	Não	Sim	Sim
Existência de membros gerenciada pela classe	Sim	Não	Não
Direcionalidade	Unidirecional	Unidirecional	Unidirecional ou bidirecional
Relacionamento	“é composto de”	“tem um”	“usa um”

DEPENDÊNCIA

Dependência

- É um relacionamento fraco e temporário entre classes
  - Uma classe depende de outra quando utiliza seus serviços ou se comunica com ela de forma momentânea
  - Não a mantêm como parte de sua estrutura interna
    - Por exemplo, uma função membro de uma classe recebe como parâmetro um objeto de outra classe, ou cria localmente esse objeto para uso pontual

Dependência

- É um relacionamento do tipo "usa um“, assim como a Associação
  - Porém o objeto é instanciado apenas quando necessário.
  - Apenas em situações pontuais, como dentro de um método

Dependência | Exemplo

- Classe **std::ostream**
  - Representa fluxos de saída, como o console.
  - Sempre que usamos o comando **cout**, estamos fazendo com que nosso código dependa temporariamente de **std::ostream** para realizar a tarefa de imprimir

```
class Logger {
public:
    void escrever(ostream& out, const string& mensagem) {
        out << "[LOG] " << mensagem << endl;
    }
};

int main() {
    Logger log;
    log.escrever(cout, "Mensagem no console");

    return 0;
}
```

Dependência

- Promove a reutilização de código e separar responsabilidades entre classes
  - Relatorio não precisa manter um objeto Formatador como atributo, apenas o utiliza momentaneamente
    - Uma classe não precisa resolver tudo sozinha
    - Delega tarefas especializadas a objetos auxiliares
    - Sem ligação duradoura

```
class Formatador {
public:
    string formatarTexto(const string& texto) {
        return "* " + texto + " *";
    }
};

class Relatorio {
public:
    void imprimir(const string& conteudo) {
        Formatador fmt;
        cout << fmt.formatarTexto(conteudo) << endl;
    }
};

int main() {
    Relatorio re;
    re.imprimir("Mensagem no console");

    return 0;
}
```

Material Complementar

- Aula 35 - POO: Relacionamentos entre objetos
  - https://youtu.be/pEIsK8ppzuo
- Aula 36 - POO: Relacionamento de Composição
  - https://youtu.be/j8yjekEIrdE
- Aula 37 - POO: Relacionamento de Agregação
  - https://youtu.be/iz9DUeBCSKo
- Aula 38 - POO: Relacionamento de Associação
  - https://youtu.be/WWaFsGlLlyE
- Aula 39 - POO: Relacionamento de Dependência
  - https://youtu.be/Fye3UxCEUlk